

Layer 6: Analysis Results

Purpose: Store the results of architectural and quality analyses performed on the system model—verification outcomes, detected issues, performance metrics, and architectural assessments.

Overview

This layer represents analysis outputs, the results of running various analytical tools and checks against the layers below. While lower layers capture the system's structure and behavior, Layer 6 is the assessment layer. It answers questions about system quality, correctness, and characteristics through formal analysis.

Key Characteristics

- All entities are results of running analyses, not extracted from code.
- Most analyses run against specific `ExecutionConfigurations`.
- Generated by HAROS analysis plugins, external tools, or custom scripts.
- Results guide development decisions, bug fixes, and optimizations.
- Analysis results tied to specific code/configuration versions.

Types of Analysis

1. **Static Analysis:** Code-level checks (unused variables, type errors, style violations).
2. **Architectural Analysis:** Graph structure checks (cycles, orphans, coupling).
3. **Communication Analysis:** QoS compatibility, rate mismatches, message type errors.
4. **Performance Analysis:** Estimated latencies, throughput, resource usage.
5. **Formal Verification:** Model checking, temporal logic verification.
6. **Compliance Checking:** Adherence to standards, best practices, project rules.

Issue

An Issue is anything that might be wrong with the system, from critical bugs to minor style violations. It represents a detected problem, violation, or warning in the system.

Fields

Field	Type	Required	Description
<code>id</code>	string	Yes	Unique identifier
<code>severity</code>	enum	Yes	Issue severity level
<code>category</code>	enum	Yes	Issue category
<code>description</code>	string	Yes	Human-readable description
<code>affected_entities</code>	list[ElementRef]	Yes	Entities involved in issue

Field	Type	Required	Description
<code>location</code>	SourceRef	No	Source code location
<code>analysis_tool</code>	string	Yes	Tool/plugin that detected issue
<code>analysis_timestamp</code>	datetime	Yes	When issue was detected
<code>execution_configuration_id</code>	string	No	Specific to configuration (if applicable)
<code>metadata</code>	map[string, any]	No	Additional context

Severity Levels

- **critical**: System cannot function correctly (e.g., type mismatch on topic)
- **error**: Likely bug or incorrect behavior (e.g., QoS incompatibility)
- **warning**: Potential problem (e.g., unused publisher, rate mismatch)
- **info**: Informational note (e.g., best practice suggestion)
- **style**: Style/formatting issue (e.g., naming convention)

Categories

- **communication**: Topic, service, action, QoS profile issues
- **timing**: Rate, latency, deadline violations
- **architecture**: Structural issues (cycles, orphans, high coupling)
- **code_quality**: Code-level problems (complexity, duplication)
- **standards_compliance**: Violation of coding standards or best practices
- **resource_usage**: Memory, CPU, bandwidth concerns
- **security**: Security vulnerabilities

Design Rationale

- **Severity classification**: Enables prioritization and filtering
- **Category taxonomy**: Groups related issues for batch fixing
- **Entity references**: Direct links to problematic elements
- **Tool tracking**: Attributes issues to analysis tools for debugging
- **Metadata flexibility**: Tool-specific context without schema changes

Example - QoS Incompatibility

```
{
  "id": "issue_qos_mismatch_001",
  "severity": "error",
  "category": "communication",
  "description": "QoS incompatibility: Publisher uses best_effort reliability, but subscriber requires reliable. Messages may be lost.",
  "affected_entities": [
    {"type": "publication", "id": "pub_sensor_data"},
    {"type": "subscription", "id": "sub_critical_controller"}
  ]
}
```

```

],
"location": {
  "file": "src/sensor_node.cpp",
  "line": 45,
  "column": 10
},
"analysis_tool": "haros_qos_checker",
"analysis_timestamp": "2025-12-08T14:30:00Z",
"execution_configuration_id": "config_production",
"metadata": {
  "publisher_qos": {
    "reliability": "best_effort"
  },
  "subscriber_qos": {
    "reliability": "reliable"
  },
  "recommendation": "Change publisher to reliable or subscriber to
best_effort"
}
}

```

Example - Orphan Publisher

```

{
  "id": "issue_orphan_pub_002",
  "severity": "warning",
  "category": "architecture",
  "description": "Publisher on topic '/debug/internal_state' has no
subscribers. This may indicate unused code or missing components.",
  "affected_entities": [
    {"type": "publication", "id": "pub_debug_state"},
    {"type": "topic", "id": "topic_debug_state"}
  ],
  "location": {
    "file": "src/controller.cpp",
    "line": 120
  },
  "analysis_tool": "haros_orphan_detector",
  "analysis_timestamp": "2025-12-08T14:30:00Z",
  "execution_configuration_id": "config_production",
  "metadata": {
    "topic_name": "/debug/internal_state",
    "node": "node_controller",
    "suggestion": "Remove if unused, or ensure subscriber exists"
  }
}

```

Metric

A Metric is a number that describes some aspect of system quality, performance, or complexity. Metrics enable trend tracking, comparison, and objective assessment.

Fields

Field	Type	Required	Description
<code>id</code>	string	Yes	Unique identifier
<code>name</code>	string	Yes	Metric name
<code>category</code>	enum	Yes	Metric category
<code>value</code>	float	Yes	Measured value
<code>unit</code>	string	No	Unit of measurement (Hz, ms, bytes, ratio)
<code>scope</code>	MetricScope	Yes	What the metric measures
<code>analysis_tool</code>	string	Yes	Tool that computed metric
<code>analysis_timestamp</code>	datetime	Yes	When metric was computed
<code>execution_configuration_id</code>	string	No	Specific to configuration (if applicable)
<code>metadata</code>	map[string, any]	No	Additional context

Categories

- **performance**: Timing, throughput, latency metrics
- **complexity**: Code/architectural complexity measures
- **quality**: Code quality indicators (coverage, duplication)
- **resource_usage**: Memory, CPU, bandwidth usage
- **architecture**: Graph metrics (coupling, cohesion, depth)
- **reliability**: Availability, fault tolerance measures

MetricScope

Field	Type	Description
<code>type</code>	enum	<code>global</code> , <code>node</code> , <code>topic</code> , <code>package</code> , <code>entity</code>
<code>entity_ref</code>	ElementRef	Reference to measured entity (if scoped)

Design Rationale

- **Flexible scope**: Measure system-wide, per-node, per-topic, or custom entities
- **Metadata for context**: Calculations, assumptions, interpretations stored
- **Versioned**: Timestamp enables trend tracking over development
- **Standardized units**: Enables comparison and aggregation

- **Rate requirements as metrics:** User requirements expressed as measurements to compare against

Example: System-Wide Metrics

```
{
  "id": "metric_total_nodes",
  "name": "total_nodes",
  "category": "architecture",
  "value": 12.0,
  "unit": "count",
  "scope": {
    "type": "global"
  },
  "analysis_tool": "haros_metrics",
  "analysis_timestamp": "2025-12-08T14:30:00Z",
  "execution_configuration_id": "config_production"
}
```

Example: Architectural Complexity

```
{
  "id": "metric_coupling",
  "name": "average_fanout",
  "category": "architecture",
  "value": 2.3,
  "unit": "ratio",
  "scope": {
    "type": "global"
  },
  "analysis_tool": "haros_architecture_analyzer",
  "analysis_timestamp": "2025-12-08T14:30:00Z",
  "execution_configuration_id": "config_production",
  "metadata": {
    "interpretation": "Average number of subscribers per publisher"
  }
}
```

VerificationResult

A **VerificationResult** captures whether a formal property or requirement holds for the system. Unlike Issues (which flag problems), a **VerificationResult** answers **yes/no questions** about system properties.

Fields

Field	Type	Required	Description
id	string	Yes	Unique identifier
property_name	string	Yes	Name of verified property
property_specification	string	Yes	Formal specification (e.g., LTL formula, natural language)
result	enum	Yes	Verification outcome
evidence	string	No	Supporting evidence (trace, proof)
affected_entities	list[ElementRef]	No	Entities involved in property
analysis_tool	string	Yes	Verification tool used
analysis_timestamp	datetime	Yes	When verification was performed
execution_configuration_id	string	No	Specific to configuration (if applicable)
metadata	map[string, any]	No	Additional context

Result Values

- **satisfied**: Property holds
- **violated**: Property does not hold
- **unknown**: Cannot determine (incomplete model, timeout)
- **not_applicable**: Property not relevant to this configuration

Design Rationale

- **Explicit outcomes**: **satisfied/violated/unknown** clarifies verification status
- **Evidence tracking**: Counterexamples or proofs aid debugging
- **Formal specifications**: Can store LTL, CTL, or natural language
- **Tool independence**: Works with various model checkers (SPIN, NuSMV, UPPAAL)

Example - Safety Property (Satisfied)

```
{
  "id": "verify_safety_001",
  "property_name": "emergency_stop_reachable",
  "property_specification": "G(emergency_signal → F(velocity = 0))",
  "result": "satisfied",
  "evidence": "All paths from emergency signal lead to zero velocity within 3 seconds",
  "affected_entities": [
    {"type": "node", "id": "node_safety_controller"},
    {"type": "topic", "id": "topic_emergency_stop"},
    {"type": "topic", "id": "topic_cmd_vel"}
  ]
}
```

```

],
"analysis_tool": "spin_model_checker",
"analysis_timestamp": "2025-12-08T14:30:00Z",
"execution_configuration_id": "config_production",
"metadata": {
  "formula_type": "LTL",
  "verification_time_seconds": 12.3
}
}

```

Example - Liveness Property (Violated)

```

{
  "id": "verify_liveness_002",
  "property_name": "sensor_data_eventually_published",
  "property_specification": "G(sensor_ready → F(data_published))",
  "result": "violated",
  "evidence": "Counterexample: Sensor ready at t=0, but no publication occurs if initialization fails. Trace: init_fail → sensor_ready=true ∧ data_published=false (forever)",
  "affected_entities": [
    {"type": "node", "id": "node_sensor"},
    {"type": "publication", "id": "pub_sensor_data"}
  ],
  "analysis_tool": "nusmv_model_checker",
  "analysis_timestamp": "2025-12-08T14:30:00Z",
  "execution_configuration_id": "config_production",
  "metadata": {
    "counterexample_trace": "init_fail → sensor_ready=true → ...",
    "fix_suggestion": "Add timeout and error handling in sensor initialization"
  }
}

```

Validation Rules

Issue Validation

1. **Valid severity:** Must be one of defined severity levels
2. **Valid category:** Must be recognized category
3. **Non-empty affected entities:** At least one entity should be referenced
4. **Tool specified:** `analysis_tool` must be provided
5. **Timestamp valid:** Must be valid ISO 8601 datetime

Metric Validation

1. **Valid category:** Must be recognized category
2. **Value type:** `value` must be numeric

3. **Positive values:** For count/rate/size metrics, `value` ≥ 0
4. **Scope consistency:** If `scope.type` is `entity`, `scope.entity_ref` must be provided
5. **Unit appropriate:** Unit should match metric type (Hz for rates, ms for latencies, etc.)

VerificationResult Validation

1. **Valid result:** Must be `satisfied`, `violated`, `unknown`, or `not_applicable`
2. **Property specified:** Both `property_name` and `property_specification` required
3. **Evidence for violations:** If `result` is `violated`, `evidence` should be provided
4. **Entity references:** All referenced entities must exist

ArchitecturalAssessment Validation

1. **Valid rating:** Must be one of defined rating levels
2. **Valid assessment type:** Must be recognized type
3. **Rationale required:** Must have non-empty rationale
4. **Recommendations encouraged:** For ratings below `good`, recommendations should be provided

Design Rationale Summary

Why Layer 6 for Analysis Results?

- **Separation of concerns:** Model structure (Layers 0-5) vs. analysis (Layer 6)
- **Versioning:** Analysis results tied to specific model versions
- **Tool independence:** Multiple tools can contribute results
- **Actionable:** Results drive development decisions

Why Various Entity Types (Issue, Metric, VerificationResult)?

- **Issues:** Problems to fix
- **Metrics:** Quantitative measurements
- **VerificationResults:** Formal property checking

Different **analysis types** need different **representation**:

- Issues: severity-based prioritization
- Metrics: numeric trend tracking
- VerificationResults: binary satisfaction checking

Why Store Analysis Results in Metamodel?

- **Historical tracking:** Compare results across versions
- **Traceability:** Link results to source entities
- **Integration:** Results inform other analyses
- **Reproducibility:** Re-run analyses, compare outcomes

Why Rate Requirements as Metrics?

- **Unified representation:** Requirements and measurements both numeric

- **Comparison:** Compare required vs. actual rates
- **Flexible:** Can add other requirement types as metrics
- **Analysis-friendly:** Enables automated requirement checking

Why Configuration-Specific Results?

- **Different configs, different issues:** Production vs. development may have different problems
- **Focused analysis:** Analyze specific deployment scenarios
- **Comparison:** Compare quality across configurations